

RankMixer：扩展工业级推荐系统中的排序模型

中文逐段翻译

ByteDance RankMixer 作者团队

Jie Zhu、Zhifang Fan、Xiaoxie Zhu、Yuchen Jiang 等

摘要

近年来，大语言模型（Large Language Models, LLMs）的进展激发了人们对扩展推荐系统规模的兴趣，但实际落地仍面临两个障碍。第一，工业级推荐系统的训练和在线服务成本必须满足严格的延迟约束，并支撑极高的 QPS。第二，排序模型中许多人工设计的特征交叉模块继承自 CPU 时代，无法充分利用现代 GPU，导致模型 FLOPs 利用率（Model FLOPs Utilization, MFU）较低，可扩展性较差。

本文提出 RankMixer，这是一种硬件感知的模型设计，目标是构建统一且可扩展的特征交互架构。RankMixer 保留了 Transformer 的高并行性，但用多头 token mixing 模块替代二次复杂度的自注意力，从而提高效率。同时，RankMixer 使用逐 token 前馈网络（Per-token FFN）同时建模不同特征子空间以及跨特征空间交互。为了获得更高 ROI，本文进一步将模型扩展为十亿参数规模的 Sparse-MoE 变体，并采用动态路由策略缓解专家训练不足和负载不均衡问题。

实验表明，RankMixer 在万亿级生产数据集上具有更优的扩展能力。通过用 RankMixer 替换此前多种人工设计且 MFU 较低的模块，模型 MFU 从 4.5% 提升到 45%，在线排序模型参数规模扩大两个数量级，而推理延迟大致保持不变。本文还在推荐和广告两个核心应用场景中进行在线 A/B 测试，验证了 RankMixer 的通用性。最终，1B dense 参数规模的 RankMixer 在全量流量中部署，在不增加服务成本的情况下，使用户活跃天数提升 0.3%，App 总使用时长提升 1.08%。

CCS 概念：信息系统 → 推荐系统。

关键词：Scaling Laws，排序模型，推荐系统。

1 引言

推荐系统（Recommender System, RS）在信息分发过程中至关重要。作为重要的机器学习应用场景，推荐系统基于大量多字段特征数据预测用户对物品的行为，这些特征包括多

种统计量等数值特征, 用户 ID 和物品 ID 等类别特征, 用户行为特征以及内容特征等 [19, 41]。当前最先进的推荐方法通常基于深度学习推荐模型 (Deep Learning Recommendation Models, DLRMs)。DLRM 在由特征得到的输入 embedding 之上使用稠密交互层, 并借助神经网络灵活捕获特征交互。DLRM 中的稠密交互层对推荐系统性能非常关键, 因此已有多种模型结构被提出 [7, 11, 33, 38, 42]。

受大语言模型通过增加参数获得性能提升的启发 [1, 14, 16], 扩展 DLRM 以充分利用海量数据已经成为迫切需求。早期关于 DLRM 扩展的研究 [2, 6, 40] 主要是在不改变结构的情况下加宽或堆叠特征交互层, 但这种方式带来的收益有限, 有时甚至会产生负面效果 [18, 32]。随后, DHEN [39] 和 Wukong [38] 等工作开始设计新的 DNN 结构, 以提升模型扩展后的性能。然而, 在推荐中利用模型规模提升性能存在独特的实际挑战。与 NLP 或视觉任务不同, 工业级推荐系统必须严格满足紧张的延迟约束, 并支持极高 QPS。因此, 关键问题是找到模型效果和计算效率之间的平衡点。

从历史上看, 推荐排序模型的结构深受 CPU 时代设计原则影响。这些模型通常依赖多种异构、人工设计的特征交叉模块来提取特征交互。但在现代 GPU 上, 许多核心算子属于 memory-bound 而不是 compute-bound, 这导致 GPU 并行性较差, MFU 往往只有个位数百分比。此外, 由于 CPU 时代模型的计算成本近似与参数数量成正比, scaling laws 所暗示的激进扩展模型规模的 ROI 很难在实践中实现。

综上, DLRM 的 scaling law 研究必须解决以下问题:

- 架构应与硬件对齐, 在现代 GPU 上最大化 MFU 和计算吞吐;
- 模型设计必须利用推荐数据特点, 例如异构特征空间, 以及数百个字段之间个性化的跨特征交互。

为解决这些挑战, 本文提出硬件感知的模型设计方法 RankMixer。其核心设计基于两个可扩展组件。第一, 多头 token mixing 仅通过无参数算子实现跨 token 特征交互, 在性能和计算效率上优于自注意力。第二, 逐 token FFN 显著扩展模型容量, 并通过为不同特征子空间分配独立参数, 缓解特征空间之间的支配问题。这些 FFN 也符合推荐数据的模式, 因此带来更好的扩展行为。

为了进一步提升大规模模型的 ROI, 本文将逐 token FFN 扩展为 Sparse Mixture-of-Experts (Sparse-MoE) 结构。通过针对不同数据的每个 token 仅动态激活部分专家, 模型能在计算成本只小幅增加的情况下显著扩大容量。RankMixer 采用类似 Transformer 的高并行架构, 但克服了基于自注意力的特征交互中的若干限制: 训练效率低、建模跨空间 ID 相似性时存在组合爆炸, 以及注意力权重矩阵带来的严重 memory-bound。同时, 在相同 FLOPs 下, RankMixer 相比 Vanilla Transformer 具有更大的模型容量和学习能力。

在抖音推荐系统的生产部署中, 本文证明在推理延迟比旧基线更短的情况下, 将模型参数扩展超过 100 倍是可行的。这得益于 RankMixer 架构能够将参数增长与 FLOPs 增长解耦, 并通过高 MFU 和工程优化进一步将 FLOPs 增长与实际服务成本解耦。

本文主要贡献如下：

- 提出 RankMixer，一种遵循硬件感知设计理念的新架构。本文设计多头 token mixing 与逐 token FFN，以高效捕获异构特征交互，并使用动态路由策略提升 RankMixer 中 Sparse-MoE 的可扩展性。
- 借助高 MFU 与性能优化手段，在不增加推理成本的情况下将模型参数扩大 70 倍，其中包括 MFU 提升和量化。
- 在万亿级工业推荐数据集上开展大量离线与在线实验，研究模型 scaling law。RankMixer 已在抖音 Feed 推荐排序中全量部署，使活跃天数和 App 使用时长分别提升 0.3% 与 1.08%。

2 相关工作

现代推荐系统主要基于 DLRM，而有效建模特征交互是 DLRM 的关键 [5, 11, 15, 22, 31, 33, 39]。Wide&Deep [5] 是早期代表工作之一，它结合逻辑回归（wide 部分）和 DNN（deep 部分），分别捕获低阶和高阶特征交互。DeepFM [11] 将因子分解机（FM）与 DNN 集成。DeepCross [26] 是残差网络 [13] 的扩展，目标是隐式学习自动特征交互。

然而，单纯依赖 DNN 学习高阶特征交互已被证明具有较大挑战 [22, 25]。显式交叉方法设计不同算子来显式捕获高阶特征交互，例如 PNN [22]、DCN [32] 及其后续 DCNv2 [33]、xDeepFM [18]、FGCNN [20] 和 FiGNN [17]。AutoInt [28] 和 Hiformer [10] 使用带残差连接的注意力机制学习复交互。DHEN [39] 提出组合多种交互算子。这些新架构虽然提升了准确率，但也增加了模型延迟和内存消耗，而且模型规模通常较小。

Scaling law 已成为深度学习中的基础主题，也是过去十年 NLP [14, 16]、CV [8, 37]、多模态建模 [23, 24] 等领域突破的重要推动因素。Scaling law 描述模型性能与模型规模、数据量、计算能力等扩展因素之间的幂律关系 [1, 16, 29, 30]。近年来，推荐系统中的 scaling law 也受到关注 [12]。已有工作探索了用户行为序列预训练 [6]、通用用户表征 [27, 40]、在线召回 [9, 34] 等方向的扩展策略。Wukong [38] 堆叠 FM 与 LCB 来学习特征交互。与之正交地，Zhang 等人 [40] 将序列推荐模型扩展到 0.8B 参数。HSTU [36] 则增强了生成式推荐器的扩展效果，其重点更多放在序列建模部分。

3 方法

3.1 整体架构

RankMixer 的整体架构由 T 个输入 token、连续 L 层 RankMixer block 以及输出池化算子组成。每个 RankMixer block 包含两个主要组件：多头 Token Mixing 层和逐 token

前馈网络 (PFFN) 层。首先, 输入向量 e_{input} 被 token 化为 T 个特征 token $x_1, x_2, \dots, x_T$, 每个 token 表示一个语义一致的特征向量。RankMixer block 通过下式在 L 层中迭代更新 token 表示:

$$S_{n-1} = \text{LN}(\text{TokenMixing}(X_{n-1}) + X_{n-1}), \quad (1)$$

$$X_n = \text{LN}(\text{PFFN}(S_{n-1}) + S_{n-1}). \quad (2)$$

其中 $\text{LN}(\cdot)$ 是层归一化函数, $\text{TokenMixing}(\cdot)$ 与 $\text{PFFN}(\cdot)$ 分别是多头 Token Mixing 模块和逐 token FFN 模块。 $X_n \in \mathbb{R}^{T \times D}$ 为第 n 个 RankMixer block 的输出, $X_0 \in \mathbb{R}^{T \times D}$ 由 $x_1, x_2, \dots, x_T$ 堆叠得到, D 为隐藏维度。输出表示 o_{output} 来自最终层表示 X_L 的均值池化, 并用于不同任务的预测。

图 1 原文为 RankMixer block 架构示意: 该 block 由多头 Token Mixing 与基于 SMOE 的逐 token FFN 组成。Token Mixing 将每个 token 的 embedding 划分为 H 个 head, 再跨 token 重组, 使不同特征之间发生交互。

图 1: RankMixer block 的架构。

3.2 输入层与特征 Token 化

构建大规模推荐模型的第一步, 是准备包含丰富信息的输入。例如, 用户特征包括用户 ID 和其他用户信息; 候选特征包括视频 ID、作者 ID 等; 序列特征经过序列模块 [4, 42] 处理后得到 e_s , 以捕获时间兴趣; 此外还包括用户与候选物品之间的交叉特征。所有特征都会被转换为不同维度的 embedding。

为了在后续阶段实现高效并行计算, 不同维度的 embedding 必须转换为维度对齐的向量, 即 feature-token。本文将这一 embedding 对齐过程称为 tokenization。最简单策略是为每个特征分配一个 embedding, 但在通常存在数百个特征的情况下会产生问题: 数百个 token 会让分配到每个 token 的参数和计算过于零碎, 导致重要特征建模不足, 并造成 GPU 核心利用不足。相反, 若 token 数过少, 例如只有一个 token, 模型会退化为简单 DNN, 无法区分多样化特征空间, 还可能让支配性特征掩盖其他特征。

为克服这些问题, 本文提出基于语义的 tokenization 方法, 结合领域知识将特征划分为若干语义一致的簇。分组后的特征先按顺序拼接为一个 embedding 向量

$$e_{\text{input}} = [e_1; e_2; \dots; e_N],$$

随后切分为合适数量、固定维度大小的 token。每个特征 token $x_i \in \mathbb{R}^D$ 捕获一组表示相近语义侧面的特征 embedding:

$$x_i = \text{Proj}(e_{\text{input}}[d(i-1) : di]), \quad i = 1, \dots, T. \quad (3)$$

其中 e_{input} 是拼接后的 embedding 向量, d 是每个 token 的固定维度, N 是特征组数量, T 是最终 token 数量, Proj 将切分后的 embedding 映射到 D 维。

3.3 RankMixer Block

3.3.1 多头 Token Mixing

为了支持特征交叉和全局信息建模, 本文引入多头 Token Mixing 模块, 以促进 token 间的信息交换。每个 token 被均匀划分为 H 个 head, token x_t 的第 h 个 head 记为 x_t^h :

$$\left[x_t^{(1)} \| x_t^{(2)} \| \cdots \| x_t^{(H)} \right] = \text{SplitHead}(x_t). \quad (4)$$

这些 head 可被看作将 x_t 投影到低维特征子空间, 因为推荐任务需要从不同视角进行建模。Token Mixing 用于融合这些子空间向量, 从而进行全局特征交互。形式化地, 多头 Token Mixing 后与第 h 个 head 对应的 token s^h 构造为

$$s^h = \text{Concat}(x_1^h, x_2^h, \dots, x_T^h). \quad (5)$$

多头 Token Mixing 模块输出 $S \in \mathbb{R}^{H \times TD/H}$, 由重新排列后的 $s^1, s^2, \dots, s^H$ 堆叠得到。本文设定 $H = T$, 以保持 Token Mixing 后 token 数量不变, 便于残差连接。经过残差连接和归一化后, 有

$$s_1, s_2, \dots, s_T = \text{LN}(\text{TokenMixing}(x_1, x_2, \dots, x_T) + (x_1, x_2, \dots, x_T)). \quad (6)$$

虽然自注意力在大语言模型中非常有效, 但本文发现它并不适合推荐系统。自注意力通过 token 内积计算注意力权重, 这适合所有 token 共享统一 embedding 空间的 NLP。但推荐任务中的特征空间天然异构。在两个异构语义空间之间计算内积相似度本身就很困难, 尤其在推荐系统中, 用户侧和物品侧特征的 ID 空间可能包含数亿个元素。因此, 对这类多样化输入使用自注意力并不优于无参数多头 Token Mixing, 反而会消耗更多计算、更多 Memory IO 以及更多 GPU 显存。

3.3.2 逐 Token FFN

此前 DLRM 和 DHEN 往往在单个交互模块中混合来自许多语义空间的特征, 这可能导致高频字段占据主导, 淹没低频或长尾信号, 最终损害推荐质量。本文引入参数隔离的前馈网络架构, 称为逐 token FFN。传统 FFN 在所有 token 间共享参数; 本文方法则为每个 token 使用专门变换, 从而隔离各 token 的参数。对于第 t 个 token s_t , 逐 token FFN 表示为

$$v_t = f_{\text{pffn}}^{t,2}(\text{Gelu}(f_{\text{pffn}}^{t,1}(s_t))), \quad (7)$$

其中

$$f_{\text{pffn}}^{t,i}(x) = xW_{\text{pffn}}^{t,i} + b_{\text{pffn}}^{t,i}. \quad (8)$$

这里 $W_{\text{pffn}}^{t,1} \in \mathbb{R}^{D \times kD}$, $b_{\text{pffn}}^{t,1} \in \mathbb{R}^{kD}$, $W_{\text{pffn}}^{t,2} \in \mathbb{R}^{kD \times D}$, $b_{\text{pffn}}^{t,2} \in \mathbb{R}^D$ 。 k 用于调整逐 token FFN 的隐藏维度, Gelu 为激活函数。

逐 token FFN 可概括为

$$v_1, v_2, \dots, v_T = \text{PFFN}(s_1, s_2, \dots, s_T). \quad (9)$$

相比所有 token 共享参数的 FFN，逐 token FFN 在保持计算复杂度不变的同时引入更多参数，因此增强了建模能力。需要强调的是，逐 token FFN 不同于 MMoE 专家：每个逐 token FFN 接收不同 token 输入，而 MMoE 中所有专家共享相同输入。换言之，MMoE 是多个专家处理同一输入，Transformer 是不同输入共享一个 FFN；RankMixer 则同时切分输入和参数，有利于学习不同特征子空间中的多样性。

3.4 RankMixer 中的 Sparse MoE

为了进一步提升扩展 ROI，可以将每个 token 的稠密 FFN 替换为 Sparse-MoE block，使模型容量增长而计算成本大致保持不变。然而，Vanilla Sparse-MoE 在 RankMixer 中会退化，原因包括：(1) 统一的 k 专家路由。Top- k 选择会平等对待所有特征 token，把预算浪费在低信息 token 上，同时让高信息 token 得不到足够专家，阻碍模型捕获 token 间差异；(2) 专家训练不足。逐 token FFN 已经按 token 数量成倍增加参数，若再添加非共享专家，专家数量会进一步爆炸，导致路由高度不均衡和专家训练不足。

本文结合两种互补训练策略解决这些问题。

ReLU Routing. 为了让 token 具有灵活专家数量并保持可微性，本文用 ReLU gate 加自适应 ℓ_1 惩罚 [35] 替代常见的 Top- k + softmax。给定 token $s_i \in \mathbb{R}^{d_h}$ 的第 j 个专家 $e_{i,j}(\cdot)$ 与路由器 $h(\cdot)$ ：

$$G_{i,j} = \text{ReLU}(h(s_i)), \quad v_i = \sum_{j=1}^{N_e} G_{i,j} e_{i,j}(s_i). \quad (10)$$

其中 N_e 是每个 token 的专家数量， N_t 是 token 数。ReLU Routing 会为高信息 token 激活更多专家，从而提升参数效率。稀疏性由 $\mathcal{L}_{\text{reg}}$ 及系数 λ 控制，使平均激活专家比例接近预算：

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{reg}}, \quad \mathcal{L}_{\text{reg}} = \sum_{i=1}^{N_t} \sum_{j=1}^{N_e} G_{i,j}. \quad (11)$$

Dense-training / Sparse-inference (DTSI-MoE). 受 [21] 启发，本文采用两个路由器 h_{train} 和 h_{infer} ，并只对 h_{infer} 施加 $\mathcal{L}_{\text{reg}}$ 。训练时两个路由器都会更新，推理时只使用 h_{infer} 。实验表明，DTSI-MoE 可以让专家不再遭受训练不足问题，同时降低推理成本。

3.5 扩展方向

RankMixer 本质上是高度并行且易扩展的架构。其参数量和计算成本可沿四个正交方向扩展：token 数量 T 、模型宽度 D 、层数 L 与专家数量 E 。对于全稠密激活版本，单

样本参数量和前向 FLOPs 可近似为

$$\#Param \approx 2kLTD^2, \quad FLOPs \approx 4kLTD^2. \quad (12)$$

其中 k 是调整 FFN 隐藏维度的比例。在 Sparse-MoE 版本中，每个 token 的有效参数和计算进一步由稀疏率

$$s = \frac{\#Activated_Param}{\#Total_Param}$$

缩放。

4 实验

4.1 实验设置

4.1.1 数据集与环境

离线实验使用抖音推荐系统训练数据。这些数据来自抖音在线日志和用户反馈标签。训练数据集包含 300 多个特征，例如数值特征、ID 特征、交叉特征和序列特征，涉及数十亿用户 ID 和数亿视频 ID，所有特征都会转换为 embedding。数据每天覆盖万亿级记录，实验使用连续两周采集的数据。

4.1.2 评估指标

为评估模型性能，本文使用 AUC 和 UAUC 作为主要性能指标，使用参数量、FLOPs 与 MFU 作为效率指标。Finish/Skip AUC/UAUC 中，finish=1/0 或 skip=1/0 标签表示用户是否完整观看视频，或是否在短时间内滑到下一个视频。本文对 finish 标签计算 AUC 与 UAUC。AUC 提升 0.0001 可视为具有较高置信度的显著提升。Dense-Param 表示稠密部分参数数量，不包括稀疏 embedding 参数。Training FLOPs/Batch 表示 batch 大小为 512 时单 batch 通过模型所需的浮点运算次数。MFU 衡量模型对硬件浮点运算能力的利用程度，等于模型实际 FLOPs 消耗除以硬件理论 FLOPs 容量。

4.1.3 基线模型

本文比较以下 SOTA 基线。DLRM-MLP 是用于特征交叉的 Vanilla MLP，也是实验基线。DCNv2 [33] 和 RDCN [3] 是特征交叉模型中的先进方法。MoE 通过多个并行专家扩大规模。AutoInt [28] 与 Hiformer [10] 结合异构自注意力层和低秩近似矩阵计算。DHEN [39] 组合多种特征交叉 block 并堆叠多层结构，包括 DCN/self-attention/FM/LR。Wukong [38] 在 DHEN 架构基础上，使用 FMB 与 LCB 研究特征交互的 scaling law。

所有实验均在混合分布式训练框架上使用数百张 GPU 进行。在该框架中，稀疏部分异步更新，稠密部分同步更新。所有模型使用一致的优化器超参数。稠密部分使用学习率 0.01 的 RMSProp，稀疏部分使用 Adagrad。

4.2 与 SOTA 方法比较

为了探索如何扩展模型，本文比较参数规模接近 100M 的模型，以确定在相同计算成本下哪种结构表现最好。表 1 总结了本文方法与基线的主要性能。结果显示，RankMixer 在多个目标和指标上显著优于其他 SOTA 模型。

表 1: 约 100M 参数推荐模型的性能与效率比较。

模型	Finish AUC↑	Finish UAUC↑	Skip AUC↑	Skip UAUC↑	参数量	FLOPs/Batch
DLRM-MLP (base)	0.8554	0.8270	0.8124	0.7294	8.7 M	52 G
DLRM-MLP-100M	+0.15%	–	+0.15%	–	95 M	185 G
DCNv2	+0.13%	+0.13%	+0.15%	+0.26%	22 M	170 G
RDCN	+0.09%	+0.12%	+0.10%	+0.22%	22.6 M	172 G
MoE	+0.09%	+0.12%	+0.08%	+0.21%	47.6 M	158 G
AutoInt	+0.10%	+0.14%	+0.12%	+0.23%	19.2 M	307 G
DHEN	+0.18%	+0.26%	+0.36%	+0.52%	22 M	158 G
HiFormer	+0.48%	–	–	–	116 M	326 G
Wukong	+0.29%	+0.29%	+0.49%	+0.65%	122 M	442 G
RankMixer-100M	+0.64%	+0.72%	+0.86%	+1.33%	107 M	233 G
RankMixer-1B	+0.95%	+1.22%	+1.25%	+1.82%	1.1 B	2.1 T

进一步观察可知，简单将 DLRM 扩展到 100M 参数只能带来有限收益，这说明需要针对推荐数据特点设计模型结构以获得更好的扩展性能。将 RankMixer 与 DCN、RDCN、AutoInt、DHEN 等经典交叉结构比较，可以看到这些模型存在参数与计算成本不平衡。即使参数规模较小，它们已经具有较大 FLOPs，限制了最终效果。RankMixer 虽然性能最佳，但扩展到 100M 参数时 FLOPs 仍相对适中，体现了模型容量和计算负载之间的平衡。与 Hiformer、Wukong 等常用扩展模型相比，在相近参数设置下，RankMixer 不仅效果更好，计算需求也更低。

4.3 不同模型的 Scaling Laws

图 2 展示了从参数规模和 FLOPs 两个角度观察到的 scaling law 曲线。RankMixer 在参数和 FLOPs 两方面都表现出最陡的 scaling law，并始终优于其他模型。

虽然 Wukong 的参数曲线较陡，但其计算成本增长更快，因此在 AUC 对 FLOPs 曲线上，它与 RankMixer、Hiformer 的差距更大。Hiformer 的性能略低于 RankMixer，说明其基于特征级 token 切分和 Attention 的方法会影响效率。DHEN 的扩展效果并不理想，

反映出其交叉结构可扩展性有限。MoE 通过增加专家扩展模型，也会面临专家平衡挑战，导致扩展表现不佳。

具体来说，RankMixer 可通过增加宽度 D 、特征 token 数 T 和层数 L 扩展。实验中观察到一个与 LLM scaling law 相同的结论：模型质量主要与总参数量相关，不同扩展方向（深度 L 、宽度 D 、token 数 T ）带来的性能几乎相同。从计算效率看，更大的隐藏维度能够产生更大的矩阵乘形状，因此相比堆叠更多层可获得更高 MFU。因此，100M 和 1B 的最终配置分别为 $(D = 768, T = 16, L = 2)$ 与 $(D = 1536, T = 32, L = 2)$ 。

4.4 消融实验

表 2: RankMixer-100M 的组件消融。

设置	ΔAUC
去掉 skip connections	−0.07%
去掉 multi-head token mixing	−0.50%
去掉 layer normalization	−0.05%
Per-token FFN $\rightarrow$ shared FFN	−0.31%

表 3: Token 到 FFN 的路由策略比较。

路由策略	ΔAUC	ΔParams	ΔFLOPs
All-Concat-MLP	−0.18%	0.0%	0.0%
All-Share	−0.25%	0.0%	0.0%
Self-Attention	−0.03%	+16%	+71.8%

在 RankMixer-100M 中，本文对残差连接、Multi-Head Token-Mixing 等组件进行消融。表 2 显示，去掉这些组件会显著降低模型性能。去掉 Multi-Head Token-Mixing 会丢失全局信息，因为每个 FFN 只能建模局部特征而没有交互。去掉残差连接和 LayerNorm 也会降低性能，因为训练稳定性下降，更容易出现梯度爆炸或消失。

本文进一步分析 token mixing 策略，也就是从特征 token 到 FFN 的路由策略。All-Concat-MLP 将所有 token 拼接后经过大 MLP，再切回相同数量 token；性能下降说明大矩阵学习困难，并削弱局部信息学习。All-Share 不切分输入，而是将整个输入向量共享并送入每个逐 token FFN，类似 MoE；其性能显著下降，说明特征子空间切分和独立建模很重要。Self-Attention 在 token 间用自注意力做路由，性能略低于 Multi-Head Token-Mixing，且计算成本更高，说明在数百个异构特征子空间之间学习相似性很困难。

4.5 Sparse-MoE 可扩展性与专家平衡

可扩展性。图 3 展示 SMOE 稀疏性与离线 AUC 增益之间的关系。Dense-Training-Sparse-Inference 与 ReLU routing 的结合，是激进稀疏下保持准确率的关键。该方法使 RankMixer 能将参数容量和内存占用扩大超过 8 倍，同时几乎不损失 AUC，并显著节省推理成本，吞吐提升超过 50%。Vanilla SMOE 在激活专家减少时性能单调下降，说明专家不平衡和训练不足问题确实存在。加入负载均衡损失可缓解退化，但仍不及 DTSI + ReLU，因为主要问题在专家训练而非路由器。这验证了 Sparse-MoE 是将 RankMixer 从当前 1B 扩展到未来 10B 规模且不突破成本预算的路径。

专家平衡与多样性。Vanilla Sparse-MoE 常出现专家不均衡，导致部分专家训练不足，最终出现“死亡专家”，即几乎不被激活的专家，而少数固定专家被持续激活。图 4 表明，DTSI 与 ReLU routing 的结合有效解决这一问题：稠密训练保证大多数专家获得足够梯度更新，防止专家饥饿；ReLU routing 使不同 token 的激活比例动态变化，能根据信息含量自适应调整，这与推荐数据多样且高度动态的分布相吻合。

4.6 在线服务成本

当参数量增加两个数量级时，如何避免推理延迟爆炸？在实际系统中，延迟与吞吐成反比，并直接关系到服务机器资源成本。相比此前全量部署的 16M 参数模型（融合 DLRM 与 DCN），RankMixer 将参数扩大大约 70 倍至 1B。尽管参数显著增加，但由于硬件对齐的模型设计和优化策略，最终推理延迟仍保持稳定。

当大幅增加模型参数时，延迟可分解为

$$\text{Latency} = \frac{\#\text{Param} \times \text{FLOPs}/\text{Param ratio}}{\text{MFU} \times \text{Theoretical Hardware FLOPs}}. \quad (13)$$

如表 6 所示，两个数量级的参数增长被逐步抵消：FLOPs/Param 比例降低 3.6 倍，MFU 提升 10 倍，基于量化的硬件 FLOPs 提升 2 倍。

表 4: 在线模型部署与成本指标。

指标	OnlineBase-16M	RankMixer-1B	变化
#Param	15.8M	1.1B	↑ 70×
FLOPs	107G	2106G	↑ 20.7×
FLOPs/Param (G/M)	6.8	1.9	↓ 3.6×
MFU	4.47%	44.57%	↑ 10×
Hardware FLOPs	fp32	fp16	↑ 2×
Latency	14.5 ms	14.3 ms	—

第一，FLOPs/Param 比例降低。由于模型设计，RankMixer 在参数增加 70 倍时 FLOPs 只增加约 20 倍，FLOPs/Param 仅为基线三分之一，即 3.6 倍效率提升。换言之，在相同 FLOPs 预算下，RankMixer 可容纳的参数量约为基线 3 倍。

第二，MFU 提升。通过使用较大的 GEMM 形状、良好并行拓扑（将并行逐 token FFN 融合到一个 kernel）、降低内存带宽成本和额外开销，RankMixer 将 MFU 提高近 10 倍，使模型从 memory-bound 转向 compute-bound。

第三，量化带来硬件 FLOPs 提升。半精度 fp16 推理会使 GPU 理论峰值 FLOPs 提高 2 倍。RankMixer 的主要计算由多个大型矩阵乘构成，适合 fp16 推理。

4.7 在线性能

为了验证 RankMixer 作为可扩展推荐模型框架的通用性，本文在个性化排序的两个核心场景进行在线实验：Feed 推荐和广告。对于每个场景，本文监控以下关键指标：

- Feed 推荐：Active Days 表示实验期间每个用户平均活跃天数，可作为 DAU 增长替代指标；Duration 表示 App 累计停留时长；Finish/Like/Comment 分别表示总完播、点赞和评论。
- 广告：报告模型质量指标 ΔAUC 和收益指标 ADVV (Advertiser Value, 广告主价值)。

这些场景中的旧基线均为结合 DLRM 与 DCN 的 16M 参数模型。本文将 Dense 部分替换为 RankMixer-1B，使 AUC 提升 0.7%。表 4 展示 Feed 推荐 8 个月长期观察的在线 A/B 测试结果，表 5 展示广告结果。

表 5: Feed 推荐在抖音与抖音极速版中的在线 A/B 测试结果，提升均具有统计显著性。

用户组	Douyin app					Douyin lite				
	Active Day	Duration	Like	Finish	Comment	Active Day	Duration	Like	Finish	Comment
Overall	+0.2908%	+1.0836%	+2.3852%	+1.9874%	+0.7886%	+0.1968%	+0.9869%	+1.1318%	+2.0744%	+1.1338%
Low-active	+1.7412%	+3.6434%	+8.1641%	+4.5393%	+2.9368%	+0.5389%	+2.1831%	+4.4486%	+3.3958%	+0.9595%
Middle-active	+0.7081%	+1.5269%	+2.5823%	+2.5062%	+1.2266%	+0.4235%	+1.9011%	+1.7491%	+2.6568%	+0.6782%
High-active	+0.1445%	+0.6259%	+1.828%	+1.4939%	+0.4151%	+0.0929%	+1.1356%	+1.8212%	+1.7683%	+2.3683%

表 6: RankMixer 在广告场景中的在线提升。

广告指标	$\Delta AUC \uparrow$	ADVV $\uparrow$
Lift	+0.73%	+3.90%

RankMixer 已经在个性化排序应用中部署和评估，包括 Feed 推荐 (RankMixer-1B) 和广告 (RankMixer-1B)。RankMixer 在所有业务关键指标上均带来统计显著提升。表 4

还显示，低活跃用户的提升最大，Active Days 提升超过 1.7412%，说明模型具有较强泛化能力。这些结果表明，RankMixer 作为统一 backbone 能够可靠泛化到不同应用场景。

5 结论

本文介绍了最新的 RankMixer 模型，该模型已经在抖音 Feed 排序中全量部署。RankMixer 将面向异构特征交互的模型设计与面向服务效率的高并行架构结合起来。实验表明，该模型具有优越性能和陡峭的 scaling law。在抖音 App 全量部署后，RankMixer 使活跃天数提升 0.3%，App 使用时长提升约 1%。

参考文献说明

原文参考文献共 42 条，包括 GPT-4、Chinchilla、Llama 2、Wide&Deep、DeepFM、DCN/ DCNv2、AutoInt、DHEN、Wukong、HSTU、DIN 等相关工作。为便于阅读，本翻译稿在正文中保留原文引用编号，参考文献题名和出版信息请以原文 PDF 为准。